Introduction to the Theory of Computation

Lecture 8. Turing Machines: Origins, Definitions, and First Programs

Neo (Yuan) Li

Oct 2026

Fudan University

yuan_li@fudan.edu.cn

Algorithms: Ancient but Undefined

Ancient Algorithm 1: Babylonian Square Root

When: c. 1800–1600 BC

Source: clay tablet YBC 7289



YBC 7289: a square of side 30, diagonal 1; 24, 51, 10 $\approx \sqrt{2}$

Method:

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{N}{x_k}\right)$$

Example: $\sqrt{2}$

x_k	value
1	1
3/2	1.5
17/12	1.41667
577/408	1.414216

Today: Newton's method

Ancient Algorithm 2: Egyptian Multiplication

When: c. 1550 BC

Source: Rhind Papyrus



Part of the Rhind Mathematical Papyrus, British Museum

Method: doubling + addition

Example: 13×21

1	21	✓
2	42	
4	84	✓
8	168	✓

$13 = (1101)_2$

$21 + 84 + 168 = 273$

Today: shift-and-add multiplication

Ancient Algorithm 3: Euclidean Algorithm

When:
c. 300 BC — Euclid, *Elements* VII
Han Dynasty — *Nine Chapters*

Method: replace (a, b) by
$(b,\ a \bmod b)$

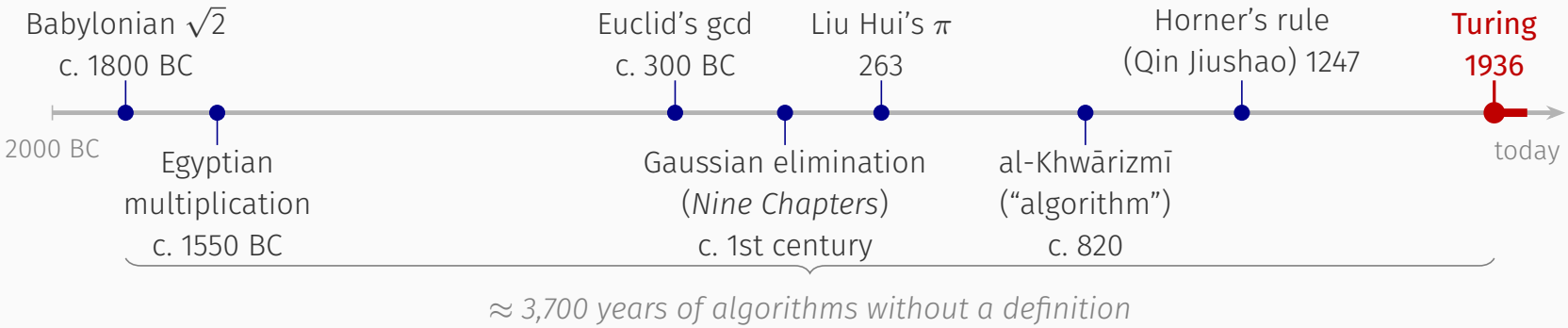
Today: RSA, modular inverse

Example: $\gcd(98, 63)$

$$\begin{aligned} 98 \bmod 63 &= 35 \\ 63 \bmod 35 &= 28 \\ 35 \bmod 28 &= 7 \\ 28 \bmod 7 &= 0 \end{aligned}$$

$\gcd = 7$

Humans have used algorithms for **thousands of years**, yet a rigorous definition came **less than a century ago**.

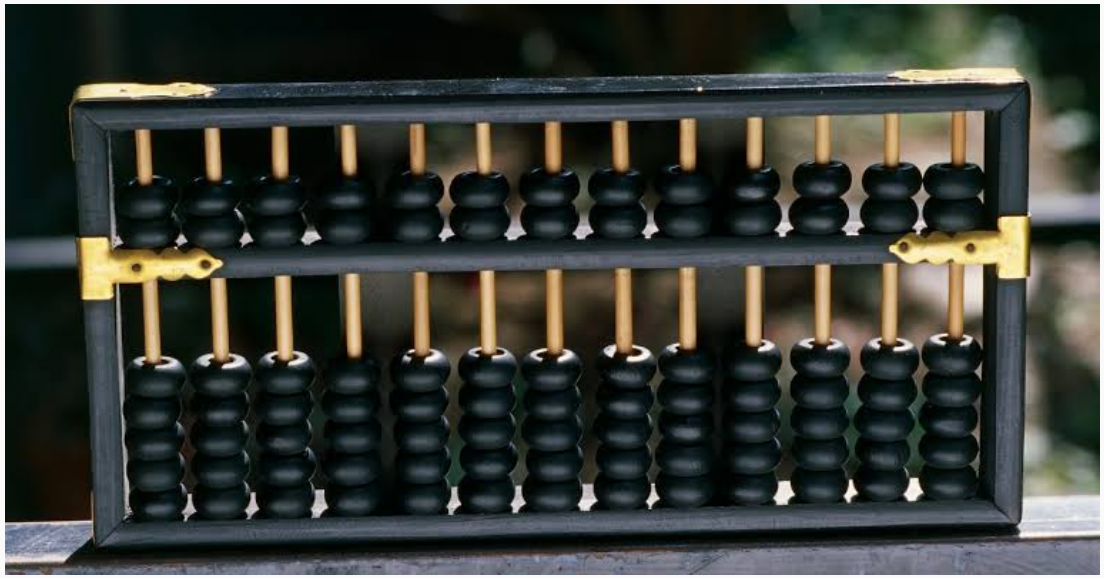


But what is an **algorithm?**

Dictionary definition: a finite sequence of precise, mechanical steps that solves a problem.

Machines: From Calculators to Computers

Early computing tools: abacus



- One of the earliest calculating aids, used for thousands of years in China, the Middle East, and Europe.
- Beads on rods represent digits; skilled users perform addition, subtraction, multiplication, and division quickly.

Early Computing Tools: 17th-Century Mechanical Calculators



Pascal's calculators, alongside a 1725 machine by Lépine (Musée des Arts et Métiers)

- Adds and subtracts with gears, with automatic carry
- Hand-powered: dial each digit with a stylus

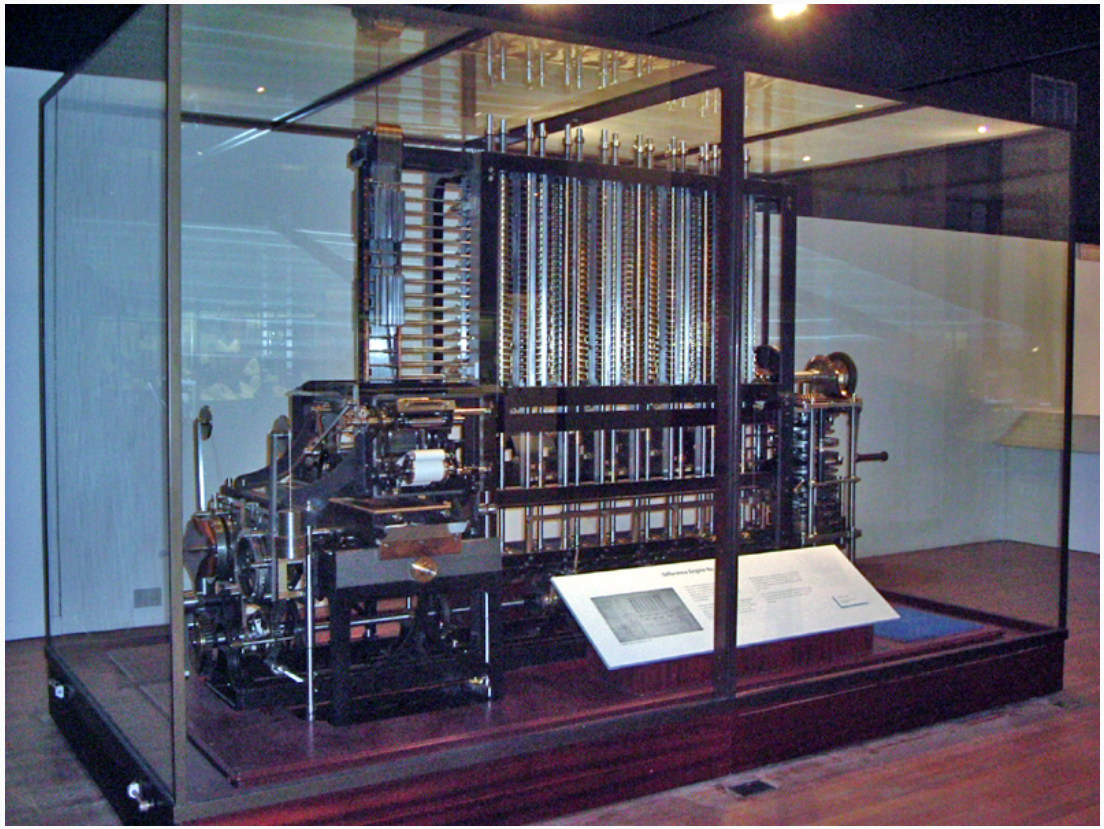
Early Computing Tools: Anton Braun's Calculator (1727)



Anton Braun's four-operation calculating machine, presented to Emperor Charles VI in Vienna, 1727

- Performs all four operations: $+$, $-$, $\times$, $\div$
- Hand-powered: set numbers, then turn a crank

Early Computing Tools: Babbage's Difference Engine



The London Science Museum's working Difference Engine No. 2, built in 1991 following Babbage's 19th-century design

- Tabulates polynomials by repeated addition
- Hand-powered: turn a crank

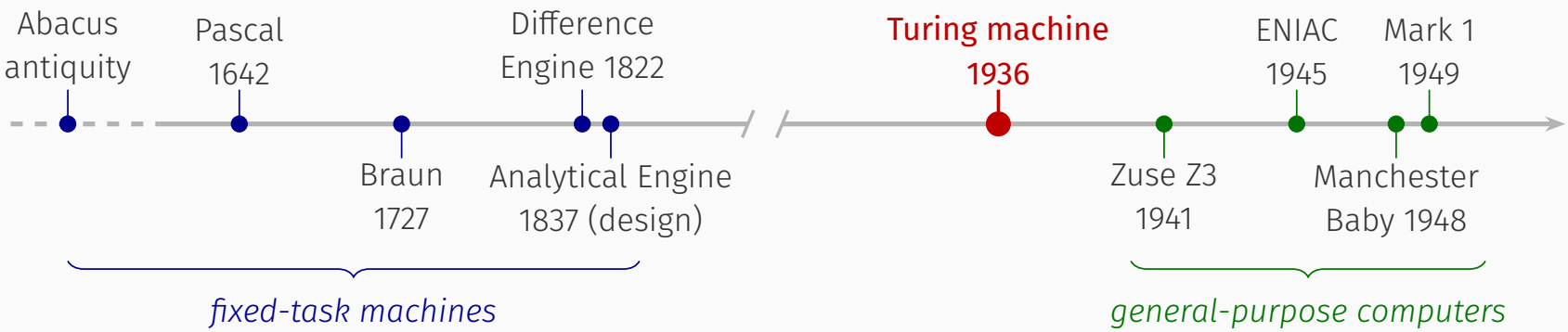
From Calculators to a General-Purpose Machine

Every machine so far performs **one fixed task**. A new task means building a new machine.

What can a machine compute, in principle? Can one machine **do it all**?

So When Was a Turing-Complete Machine Actually Built?

The Turing machine (1936) was purely theoretical.
Engineering caught up about a decade later.



Turing designed the ACE (1945) and wrote the programming manual for the Manchester computers.

Mathematics: Hilbert's Question

Hilbert's Program

Crisis (c. 1900):
paradoxes in the foundations of mathematics, e.g. Russell's paradox (1901)

Hilbert's answer (1920s):
formalize all of mathematics

- a fixed list of axioms and rules of inference
- a proof = a finite string of symbols, **checkable mechanically**

Goal: show it is **complete**, **consistent**, and **decidable**



Hilbert in 1912

"Wir müssen wissen — wir werden wissen."

We must know — we will know. — David Hilbert, 1930

Gödel's Incompleteness Theorems (1931)

Any consistent formal system strong enough for arithmetic:

1. contains true statements it **cannot prove**
2. **cannot prove its own consistency**

Hilbert's goal	Answer	
Complete?	**No**	Gödel, 1931
Consistent (provably)?	**No**	Gödel, 1931
Decidable?	???	open

The Entscheidungsproblem (Hilbert & Ackermann, 1928)

*Is there an **algorithm** that, given any statement of first-order logic, decides whether it is provable?*

To answer "Yes":
write down one algorithm.

To answer "No":
rule out *every possible* algorithm
$\Rightarrow$ need a precise definition of "algorithm".

1935: Turing learns of it in Max Newman's lectures

1936: Turing, *On Computable Numbers, with an Application to the Entscheidungsproblem*
(independently: Church, 1936)

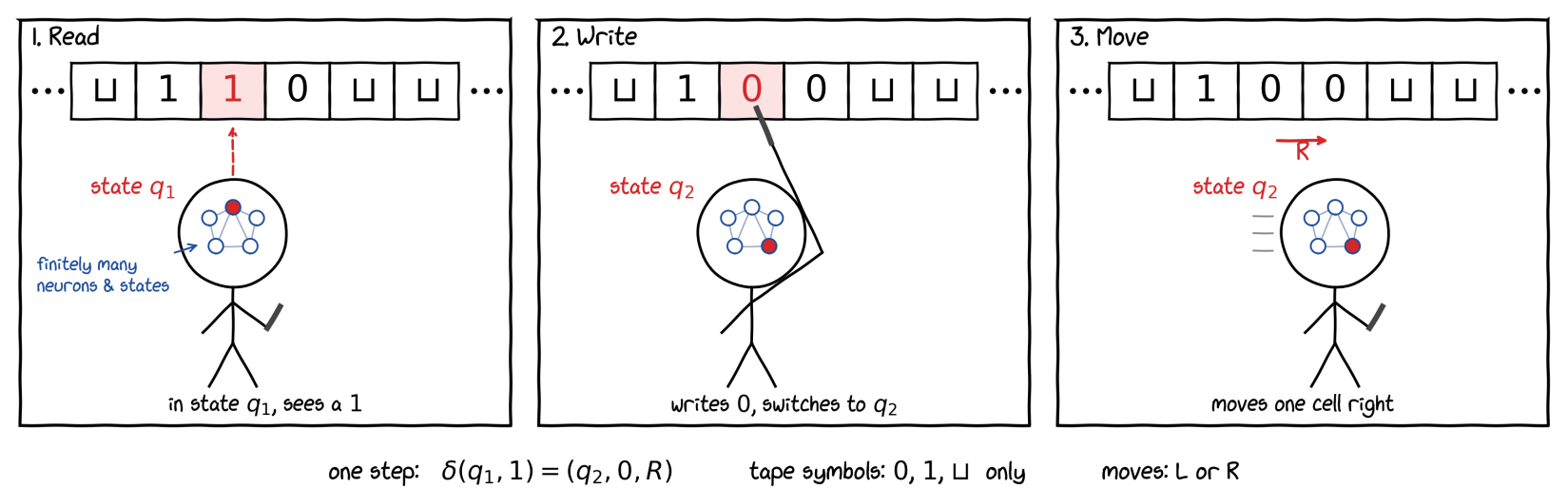
Answer: No.

The Turing Machine: Formal Definition

Definition 1 (Turing machine)
A *Turing machine* is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q, Σ, Γ are all finite sets and

1. Q is the set of states,
2. Σ is the input alphabet not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_{\text{accept}} \in Q$ is the accept state, and
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

The Turing Machine: A Human at Work



Each step of the computation has three parts:

1. **Read** the symbol in the current cell.
2. **Write** a new symbol there, and **update** the state.
3. **Move** the head one cell left or right.

The Turing Machine: Computing a Function

Definition 2 (Turing machine, computational version)
A *computational Turing machine* is a 6-tuple

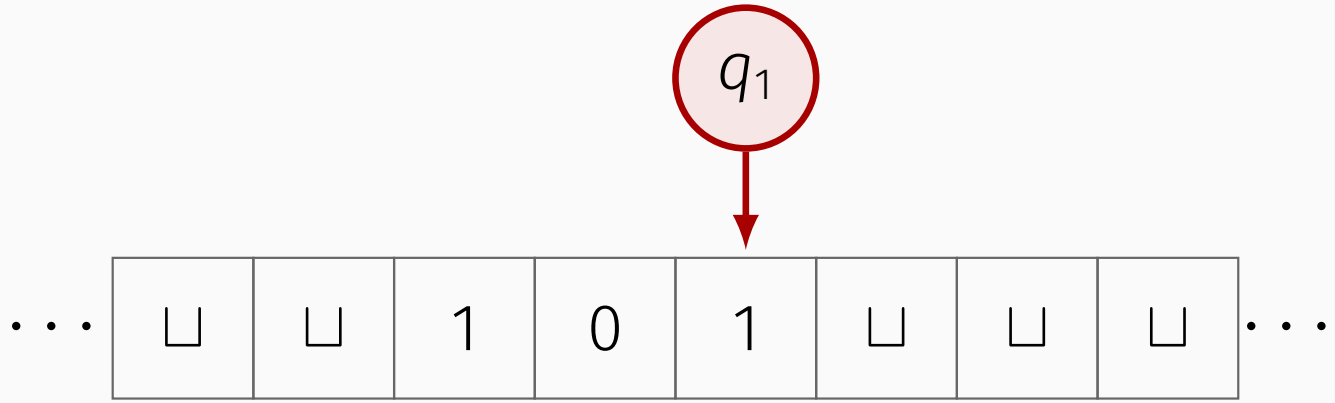
$$(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{end}}),$$

where Q, Σ, Γ are all finite sets and

1. Q is the set of states,
2. Σ is the input alphabet not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $q_{\text{end}} \in Q$ is the end state.

M computes $f : \Sigma^* \to \Sigma^*$ if, on every input $w \in \Sigma^*$, M halts in q_{end} with $f(w)$ on its tape.

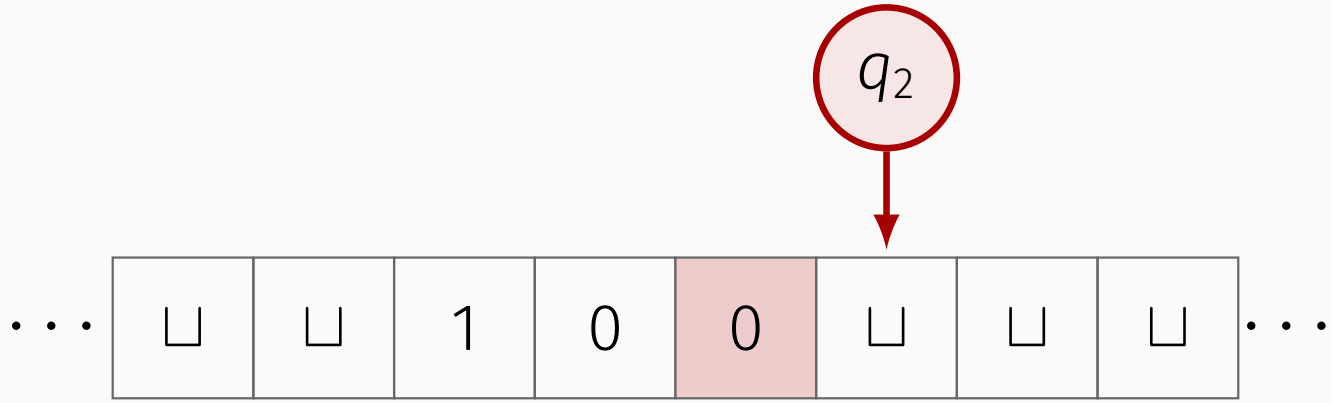
A Turing Machine at Runtime (1/3)



Read 1 $\Rightarrow$ write 0, move right, go to q_2

$$\delta(q_1, 1) = (q_2, 0, R)$$

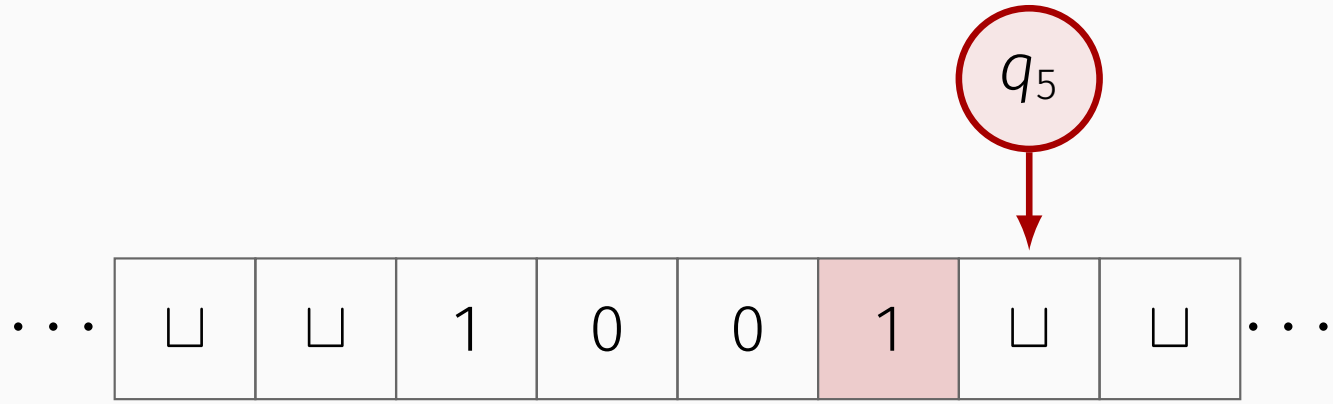
A Turing Machine at Runtime (2/3)



Read $\sqcup$ $\Rightarrow$ write 1, move right, go to q_5

$$\delta(q_2, \sqcup) = (q_5, 1, R)$$

A Turing Machine at Runtime (3/3)

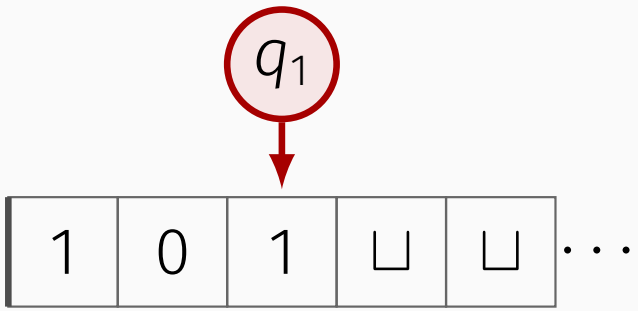


The blank has been overwritten; the machine is now in q_5

The Turing Machine: Configurations

Definition 3 (Configuration)
A *configuration* of M records everything needed to continue the computation: the current state, the full tape contents, and the head position.

Formally, we write a configuration as $u\,q\,v$, where $q \in Q$ is the current state, $u, v \in \Gamma^*$, the tape contains uv followed by blanks, and the head is on the first symbol of v.



Configuration:

$$\underbrace{10}_{u}\ q_1\ \underbrace{1}_{v}$$

The Turing Machine: What Does It Mean to Accept?

Definition 4 (Acceptance)
M accepts input w if there exists a sequence of configurations $C_0, C_1, \ldots, C_k$ such that:

1. C_0 is the *start configuration*: state q_0, with w written on the tape and the head at the leftmost cell,
2. each C_i *yields* C_{i+1} — that is, C_{i+1} follows from C_i by one application of δ, and
3. C_k is an *accepting configuration*: its state is q_{accept}.

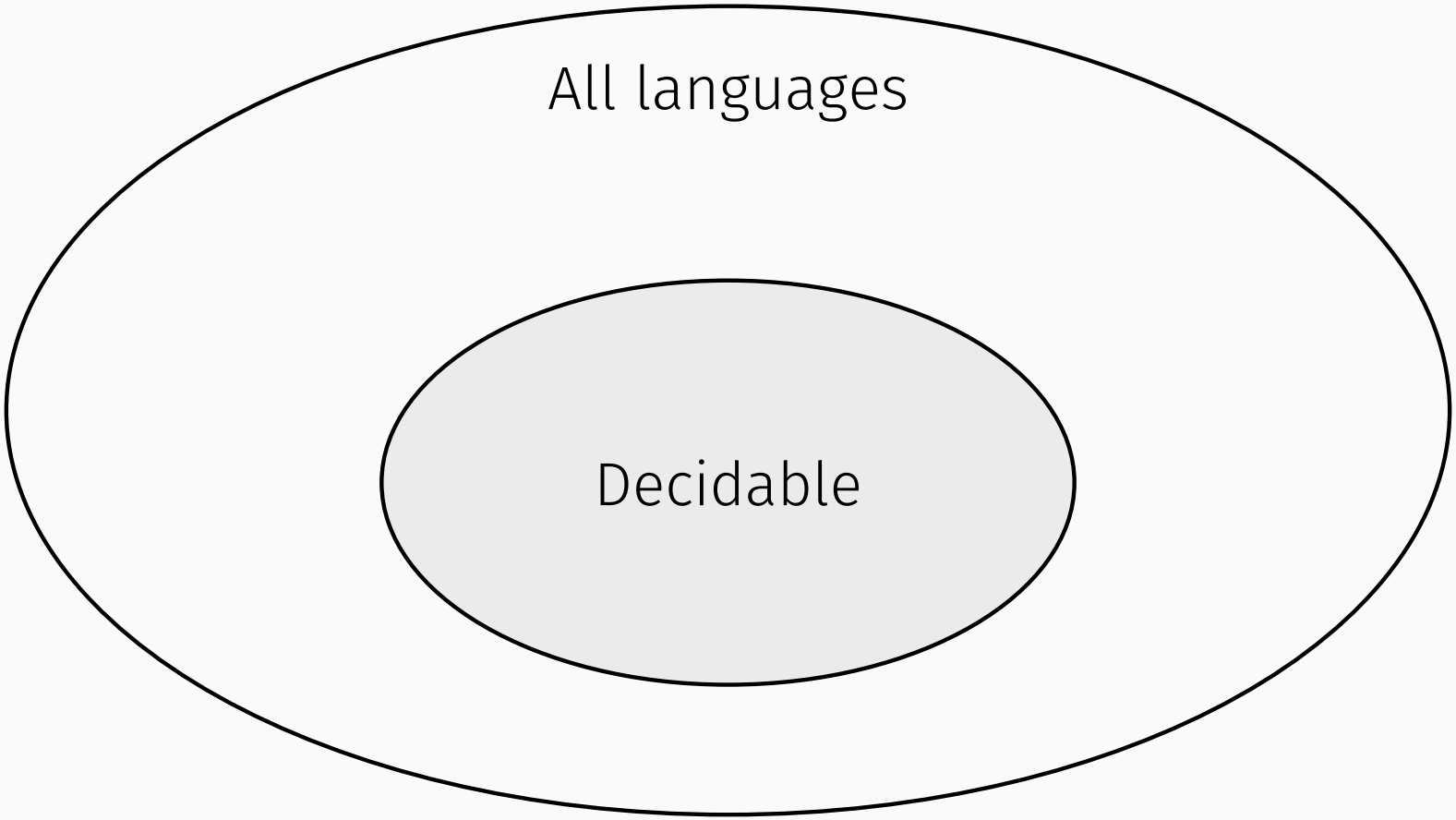
Deciding a Language

Definition 5
Let $L \subseteq \{0,1\}^*$ be a language, M a Turing machine, and $T : \mathbb{N} \to \mathbb{N}$. We say that M *decides* L *in time* $T(n)$ if for every $x \in \{0,1\}^*$,

1. M halts within $T(|x|)$ steps,
2. if $x \in L$, then M accepts x, and
3. if $x \notin L$, then M rejects x.

Definition 6
Language L is *decidable* if some TM decides L.

Decidable Languages



L is **decidable** if some TM halts on every input and accepts exactly the strings in *L*.

Accept, Reject, or Loop Forever

Note. On an input x, a Turing machine may **accept**, **reject**, or **loop forever**.

If a TM *decides* a language, it halts on every input — it never loops forever.

Definition 7
The set of strings that a Turing machine M accepts is called the language *recognized by* M, denoted $L(M)$.

Decidable vs. Recognizable

Definition 8
Let $L \subseteq \{0,1\}^*$ be a language. We call L *recognizable* if there is a Turing machine M such that $L(M) = L$.

Every decidable language is recognizable; the converse is **not** true. For example, the halting problem is recognizable but not decidable.

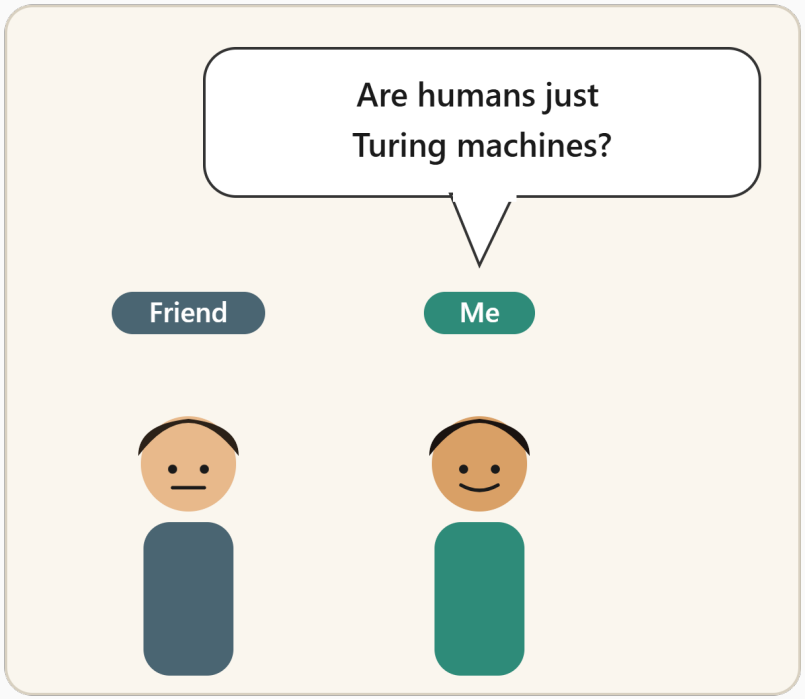
Computable Function

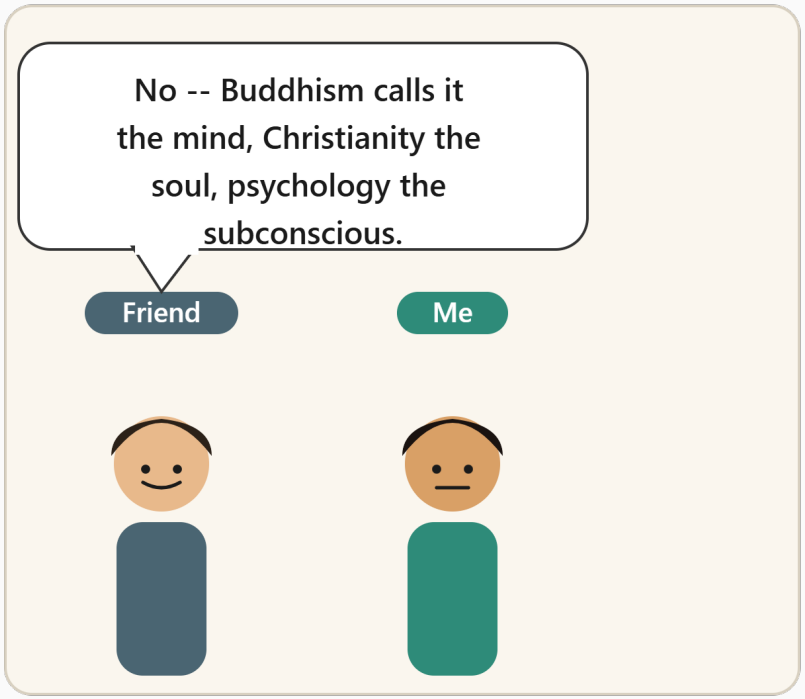
Definition 9 (Computable function)
Let $f : \{0,1\}^* \to \{0,1\}^* \cup \{\perp\}$ be a (possibly partial) function, where $\perp$ denotes "undefined," and let $T : \mathbb{N} \to \mathbb{N}$.

A Turing machine M *computes* f *within time* $T(n)$ if for every $x \in \{0,1\}^*$ with $f(x) \neq \perp$, M halts on x within $T(|x|)$ steps, with exactly $f(x)$ on its tape.

f is *computable* if some TM computes it.

Are Humans Turing Machines?





Turing Machine Programming

Example: Binary Add One

1). Binary add one

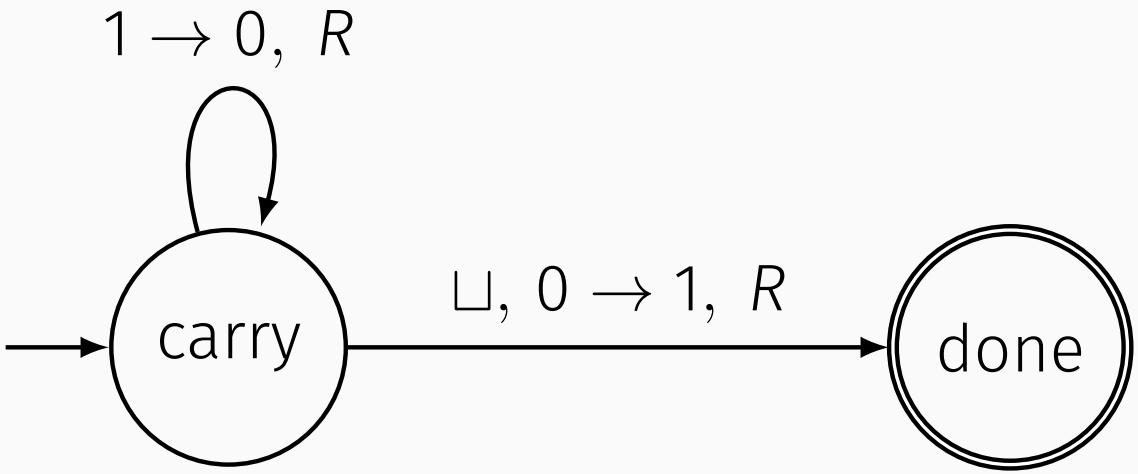
- **Input**: a non-negative binary number n, least significant bit first.
- **Output**: $n + 1$.

For example:

Input:	101␣
Output:	011␣

The input alphabet $\Sigma = \{0, 1\}$, and the tape alphabet is $\Gamma = \{0, 1, ␣\}$.

Example: Binary Add One



Turing machine: binary add one (LSB first)

Simulating It: turingmachinesimulator.com

```
name: binary add 1, LSB first
init: q0
accept: q_end

q0,1
q0,0,>

q0,0
q_end,1,-

q0,_
q_end,1,-
```

Example: Binary Add Two

2). Binary add two

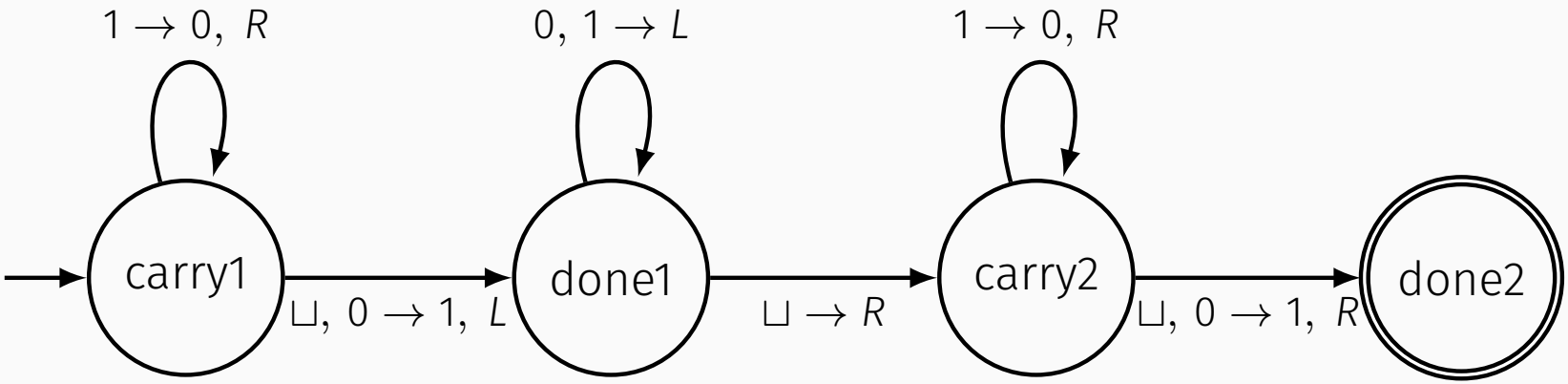
- **Input**: a non-negative binary number n, least significant bit first
- **Output**: $n + 2$

For example:

Input: 001␣
Output: 011␣

Approach 1: Reuse the Add-One Machine, Twice

We can copy and paste the add-one "code." That is, run the add-one machine, rewind the head back to the start, and run it again.



Turing machine: binary add two (LSB first)

Simulating It: Approach 1

```
name: binary_add_two_lsb
init: carry1
accept: done2

carry1,1
carry1,0,>

carry1,0
done1,1,<

carry1,_
done1,1,<

done1,0
done1,0,<
```

```
done1,1
done1,1,<

done1,_
carry2,_,>

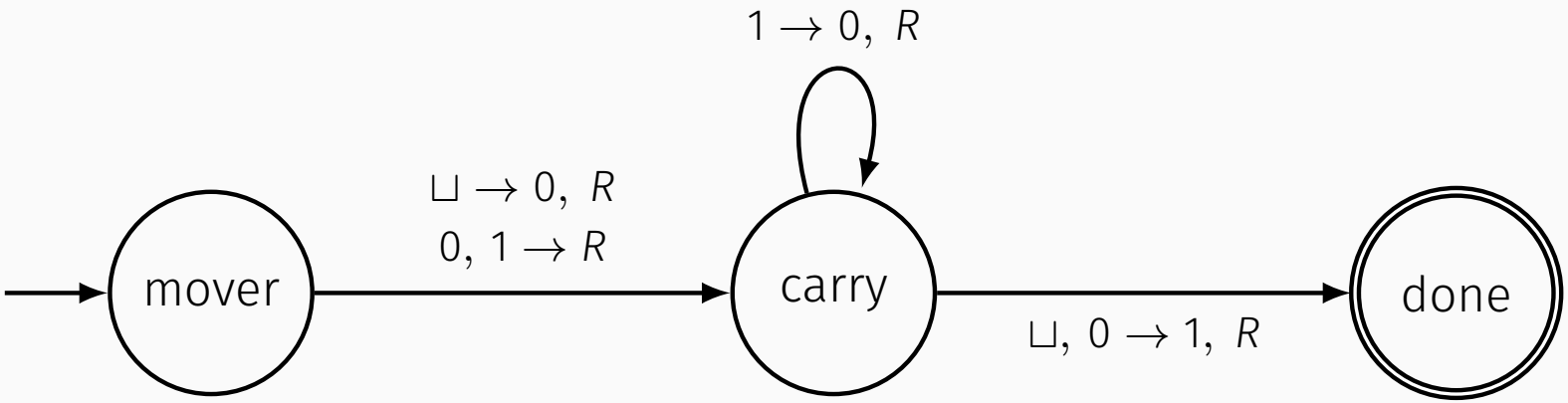
carry2,1
carry2,0,>

carry2,0
done2,1,>

carry2,_
done2,1,>
```

Approach 2: Shift, Then Add One

Or, we can move the head right once, and then perform add one.



Turing machine: binary add two (LSB first)

Simulating It: Approach 2

```
name:
    binary_add_two_lsb_mover

init: mover
accept: done

mover,0
carry,0,>

mover,1
carry,1,>

mover,_
carry,0,>
```

```
carry,1
carry,0,>

carry,0
done,1,>

carry,_
done,1,>
```

Exercises: Turing Machine Programming

Design a Turing machine for each task.

1. **Unary add one:** $0^k \mapsto 0^{k+1}$.
   e.g. $000 \mapsto 0000$

2. **Duplicate zeros:** $0^k \mapsto 0^{2k}$.
   e.g. $00 \mapsto 0000$

3. **Multiply by 2:** $x \mapsto 2x$, where $x \in \{0,1\}^*$ is written least significant bit first.
   e.g. $011 \mapsto 0011$